

halfedge：各向同性重网格化模块设计文档

src/remesh.rs

halfedge 库

2026-07-04

目录

1	模块定位	1
1.1	API 总览	2
2	RemeshStats	2
3	核心算法：4 步迭代	2
3.1	步骤 1：分裂长边 <code>split_long_edges</code>	2
3.2	步骤 2：折叠短边 <code>collapse_short_edges</code>	2
3.3	步骤 3：翻转均衡度数 <code>flip_for_valence</code>	3
3.4	步骤 4：切向拉普拉斯平滑 <code>smooth_vertices</code>	3
3.5	整体流程	4
4	目标长度自动估计	4
5	阈值与常数	4
6	复杂度	4
7	测试覆盖	5
8	设计取舍	5

1 模块定位

`remesh.rs` 实现各向同性重网格化 (isotropic remeshing)：通过迭代式分裂长边、折叠短边、翻转边均衡度数、切向拉普拉斯平滑，将网格边长驱向目标长度，生成均匀三角形。

参考：Botsch & Kobbelt, “A Remeshing Approach to Multiresolution Modeling” (2004)。

1.1 API 总览

类别	API	语义
核心	<code>isotropic_remesh(mesh, target, iter, reproject, preserve_boundary)</code>	4 步迭代流水线
快捷	<code>quick_remesh(mesh)</code>	自动目标长度 + 3 次迭代
快捷	<code>remesh_to_length(mesh, L)</code>	指定目标长度 + 5 次迭代
统计	<code>RemeshStats</code>	迭代/分裂/折叠/翻转次数与目标长度

2 RemeshStats

字段	类型	含义
<code>iterations</code>	<code>usize</code>	总迭代次数
<code>splits</code>	<code>usize</code>	分裂的边数
<code>collapses</code>	<code>usize</code>	折叠的边数
<code>flips</code>	<code>usize</code>	翻转的边数
<code>target_length</code>	<code>f64</code>	实际使用目标长度

3 核心算法：4 步迭代

每轮迭代依次执行：

3.1 步骤 1：分裂长边 `split_long_edges`

阈值 $L_{\max} = \frac{4}{3}L$ 。遍历所有规范半边（twin 对中 ID 较小者），若 `edge_length(h) > Lmax`，调用 `split_edge`（在中点插入顶点）。涉及非流形端点的边**跳过**（见「非流形防护」）。边界边**跳过**：分裂边界边会在边界环上插入中点、增加边界顶点，改变边界拓扑（见「边界保持」）。

3.2 步骤 2：折叠短边 `collapse_short_edges`

阈值 $L_{\min} = \frac{4}{5}L$ 。候选需满足：

$$L_{\min} > \ell(h) > 10^{-10}.$$

折叠目标位置 = 边中点（无 twin 的边界边折叠到 tip 当前位置），调用 `collapse_edge_at`（保留 QEM 接口的可定制位置）。涉及边界顶点的边**跳过**（见「边界保持」）。

非流形防护 每轮迭代开始先扫描网格得到非流形顶点集合（判定依据：outgoing 环绕长度 $\neq$ 真实入射半边数）。`split` / `collapse` / `flip` 三个阶段对涉及非流形端点的边**统一跳过**，避免 `VertexRing` 只遍历一个扇区导致链接条件误判、残留悬空引用、破坏拓扑。`collapse` 阶段还会在每次折叠后重新校验合并出的新顶点，若其变为非流形则同步加入跳过集合。

边界保持 参数 `preserve_boundary` (默认 `true`) 控制是否保持开放网格的边界不变: `split` 跳过边界边 (`is_boundary_edge`), 避免在边界环上插入中点; `collapse` 跳过任一端点为边界顶点的边 (`edge_touches_boundary`), 既覆盖边界边, 也覆盖「一个边界 + 一个内部」的内部边——折叠后者会移动/合并边界顶点、破坏边界环。 `flip` 本就要求两侧都有面 (边界边不参与), `smooth` 本就跳过边界顶点, 故无需额外处理。设为 `false` 时允许 `split/collapse` 修改边界 (拓扑操作本身会维护边界环的 `next/prev` 链, 见 `split_edge` 文档)。

表面投影 (reproject) 当 `reproject = true` 时, 迭代前快照原始表面为三角形列表; 每轮平滑后, 对每个顶点求其到原始表面的最近点 (点到三角面最近点, 暴力 $O(VF)$) 并投影回该点, 避免切向平滑导致体积收缩/形状漂移。

3.3 步骤 3: 翻转均衡度数 `flip_for_valence`

对每个内部边, 相邻两三角形四顶点 a, b, c, d , 目标度数:

$$t_v = \begin{cases} 6 & v \text{ 内部} \\ 4 & v \text{ 边界} \end{cases}$$

翻转前后度数偏差和:

$$\text{before} = |val_a - t_a| + |val_b - t_b| + |val_c - t_c| + |val_d - t_d| \quad (1)$$

$$\text{after} = |val_a - 1 - t_a| + |val_b - 1 - t_b| + |val_c + 1 - t_c| + |val_d + 1 - t_d| \quad (2)$$

仅当 `after < before` (严格改善) 时翻转, 避免循环。

翻转合法性: 对角线已相连检测 翻转会把对角线 (a, b) 换成 (c, d) 。若 c, d 已经相连, 翻转后同一条边会被 4 个面共享, 产生非流形边。因此**翻转前**先检测 c, d 之间是否已存在边 (复杂度 $O(\deg)$), 已相连则跳过该翻转。

边界半边过滤 遍历半边时需过滤边界半边: 仅当 $h.\text{twin} \neq \text{None}$ 且 $h.\text{face} \neq \text{None}$ 且 $h.\text{twin}.\text{face} \neq \text{None}$ 时才进入翻转判定。开放网格的边界半边满足 $\text{twin} = \text{Some}$, $\text{face} = \text{None}$, 若直接 `unwrap h.face` 会 panic。被跳过的边界半边其内部 `twin` 会在另一轮迭代中处理 (满足 $\text{face} = \text{Some}$)。

3.4 步骤 4: 切向拉普拉斯平滑 `smooth_vertices`

跳过边界顶点。对内部顶点 v :

$$a\vec{v}g = \frac{1}{|N(v)|} \sum_{u \in N(v)} \vec{p}_u \quad (3)$$

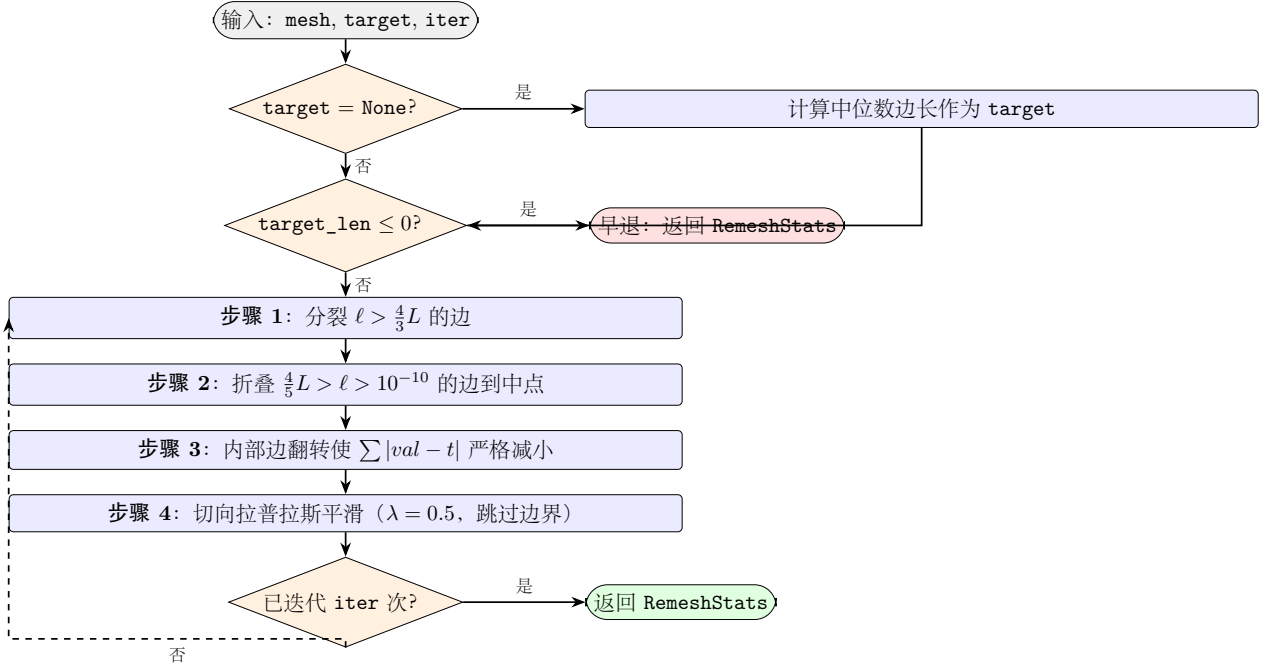
$$\vec{n} = \text{vertex_normal}(v) \quad (\text{None 时回退}(0, 1, 0)) \quad (4)$$

$$\vec{t} = (a\vec{v}g - \vec{p}_v) - \vec{n} \left((a\vec{v}g - \vec{p}_v) \cdot \vec{n} \right) \quad (5)$$

$$\vec{p}_v \leftarrow \vec{p}_v + \lambda \vec{t}, \quad \lambda = 0.5 \quad (6)$$

$\vec{t}$ 是位移在切平面的投影, 保证顶点沿曲面滑动而非偏离。

3.5 整体流程



4 目标长度自动估计

`compute_target_length` (`target = None` 时调用): 收集所有半边长度, 排序后取 $[n/4, 3n/4]$ 区间均值为**截尾均值** (trimmed mean), 抑制退化边 / 边界短边对均值的拉低。

5 阈值与常数

常数	值	用途
$L_{\max}$	$\frac{4}{3}L$	分裂阈值
$L_{\min}$	$\frac{4}{5}L$	折叠阈值
λ	0.5	切向平滑松弛因子
退化阈值	10^{-10}	折叠时边长下限 (防零长边)
目标度数 (内)	6	三角网格规则内部度数
目标度数 (边)	4	三角网格规则边界度数
截尾均值区间	$[n/4, 3n/4]$	<code>compute_target_length</code>
法向回退	(0, 1, 0)	<code>vertex_normal</code> 返回 <code>None</code> 时

6 复杂度

设 V, E, F 为顶点 / 半边 / 面数, I 为迭代次数, $\bar{d}$ 为平均度数 (三角网格约 6)。

函数	时间复杂度
<code>compute_target_length</code>	$O(E \log E)$ (排序)
<code>split_long_edges</code>	$O(E)$
<code>collapse_short_edges</code>	$O(E)$
<code>flip_valence</code>	$O(E \cdot \bar{d}) \approx O(E)$ (每边 4 次 <code>VertexRing::count</code>)
<code>smooth_vertices</code>	$O(V \cdot \bar{d}) \approx O(V)$
<code>isotropic_remesh</code>	$O(I \cdot (E + V))$
<code>quick_remesh</code>	$O(3 \cdot (E + V))$
<code>remesh_to_length</code>	$O(5 \cdot (E + V))$

7 测试覆盖

测试名	验证点
<code>compute_target_length_icosphere</code>	自动目标长度 > 0 且 < 2.0 (单位球)
<code>remesh_icosphere_preserves_topology</code>	<code>quick_remesh</code> 后 <code>validate_mesh</code> 通过, $F \in [0.5F_0, 2F_0]$
<code>remesh_preserves_closedness</code>	重网格化后 icosphere 仍闭合
<code>target_valence_interior_is_6</code>	icosphere 所有内部顶点目标度数 = 6
<code>remesh_to_specific_length</code>	<code>remesh_to_length(0.5)</code> 后校验通过
<code>isotropic_remesh_zero_target_length_returns_early</code>	<code>target=0</code> 早退: <code>iterations</code> 报告请求值, 操作计数全 0
<code>isotropic_remesh_negative_target_length_returns_early</code>	<code>target=-1</code> 早退: 同上
<code>compute_target_length_empty_mesh_returns_zero</code>	空网格目标长度 = 0 (无半边)
<code>remesh_on_open_grid_does_not_panic</code>	<code>build_grid</code> 开放网格重网格化不 panic, 校验通过
<code>remesh_preserves_boundary_vertices_on_open_grid</code>	开放网格 <code>split/collapse</code> 均不改动边界顶点集合
<code>remesh_preserve_boundary_false_allows_boundary_split</code>	<code>preserve_boundary=false</code> 允许分裂边界边
<code>remesh_reproject_keeps_vertices_on_unit_sphere</code>	<code>reproject=true</code> 顶点投影回单位球面 (半径 ≈ 1)
<code>target_valence_boundary_is_4</code>	开放网格边界顶点目标度数 = 4, 内部 = 6
<code>tangential_smooth_on_isolated_vertex_is_noop</code>	孤立顶点切向平滑不 panic, 位置不变
<code>isotropic_remesh_zero_iterations_is_noop</code>	<code>iter=0</code> 所有计数为 0
<code>remesh_preserves_euler_characteristic</code>	icosphere 重网格化前后 $\chi = 2$ 不变
<code>nonmanifold_vertices_detects_pinch</code>	共享顶点的两个四面体: 识别 pinch 顶点为非流形
<code>remesh_collapse_skips_nonmanifold_endpoints</code>	折叠跳过非流形端点后, 不引入非流形边/悬空引用
<code>remesh_split_flip_skip_nonmanifold_vertices</code>	<code>split/flip</code> 同样跳过非流形顶点, 重网格化后拓扑保持
<code>are_vertices_connected_basic</code>	两顶点间是否已存在边的判定正确

全模块共 17 个单元测试, 全库共 552 个。

8 设计取舍

- 为何用中位数边长而非均值? 中位数对退化边 / 异常长边鲁棒, 截尾均值进一步抑制极端值, 给出符合直觉的「代表性边长」。

- **为何翻转条件是严格改善？** 等价情况翻转会陷入循环 ($A \rightarrow B \rightarrow A \rightarrow \dots$)，严格 **after** < **before** 保证单调下降。
- **为何切向平滑而非各向同性平滑？** 各向同性平滑会让顶点沿法向漂移，导致网格「收缩」。切向投影 $\vec{t} = \vec{d} - (\vec{d} \cdot \vec{n})\vec{n}$ 保持顶点在原曲面附近，几何形状保持更好。
- **为何跳过边界顶点？** 边界曲线是网格的「骨架」，移动会改变网格边界，破坏与原始数据的对应关系。开放网格重网格化应保持边界不变。
- **为何 reproject 参数当前是 no-op？** 真正的再投影需要参考原始曲面（如 BVH + 最近点查询），实现复杂。当前切向平滑已能保持形状，再投影留作未来扩展占位。
- **为何规范半边用 ID 比较？** Twin 对 (h, t) 中只处理一个， $\min(h, t)$ 是确定的、无需额外状态。依赖 SlotMap 的 ID 唯一性不变量。
- **为何 split/collapse/flip 都跳过非流形顶点？** VertexRing 只能遍历非流形顶点的其中一个扇区，会令链接条件误判并残留悬空引用；三段统一跳过涉及非流形端点的操作，即可保证重网格化不破坏拓扑。collapse 还额外对合并出的新顶点做二次校验，覆盖折叠过程中新产生的非流形顶点。